NAME: TANKISO MASOEBE

COMPUTER SECURY W10

1. Snort Rule Mastery: Research and Analysis

1. Introduction to Snort

Snort is an open-source Intrusion Detection and Prevention System (IDS/IPS) used to monitor network traffic, identify suspicious activity, and help protect computer systems from cyberattacks. An IDS detects and reports suspicious traffic, while an IPS can also block malicious traffic when configured to operate inline. Snort analyses packets using rules thatscribe specific patterns, protocols, addresses, ports, and suspicious behaviours.

Network Intrusion Detection & Prevention System
+1

2. Understanding Snort rule syntax

A Snort rule generally consists of two main sections: a rule header and a rule body. The header specifies the action, protocol, source and destination addresses, ports, and direction of traffic. The body contains options that define the conditions that must be satisfied for the rule to trigger an alert.

Snort 3 Rule Writing Guide
+1

Example:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (
   msg:"Suspicious HTTP traffic detected";
   content:"malicious";
   sid:1000001;
   rev:1;
)
```

Explanation of the components:

- alert — generates an alert when the rule matches.

- tcp — examines TCP traffic.
- $EXTERNAL_NET — represents the external network.
- any — allows any source port.
- -> — specifies traffic moving in one direction.
- $HOME_NET 80 — targets the internal network on destination port 80.
- msg — provides the alert description.
- content — searches for the specified text in the relevant packet data.
- sid — identifies the rule with a unique signature ID.
- rev — indicates the rule's revision number.

The example is illustrative rather than a verified Emerging Threats signature. In a real deployment, the rule must be tested against the intended traffic and the syntax supported by the installed Snort version.

Snort 3 Rule Writing Guide
+1

3. Analysis of three existing Snort rules

The following rules are documented in the official Snort rule database. Each example is analysed in terms of its intended attack detection and the logic behind the signature.

Rule Docs 1:13513
+2

Rule 1: Generic SQL injection using the INSERT command

Source: Snort Rule 1:13513

Alert message: SQL generic SQL insert injection attempt — GET parameter

Summary (approximately 200 words)

This rule is designed to detect potential SQL injection attacks in HTTP GET parameters. SQL injection occurs when an attacker places malicious SQL statements into input fields or URL parameters that a web application uses to construct database queries. If the application does not validate input correctly, the attacker may manipulate database operations, access confidential information, modify records, or interfere with normal application functions.

The Snort rule database identifies this signature as a generic SQL INSERT injection attempt. Its purpose is to recognise traffic associated with the insertion of SQL code

through a GET request. A GET request commonly carries information in the URL query string, making its parameters a possible target for malicious input.

The rule operates by examining network traffic against the signature's defined conditions. When the relevant SQL injection pattern is detected in the inspected request, Snort generates an alert identifying the suspicious activity. The signature does not prove that the attack succeeded; it indicates that traffic resembles a known attack pattern.

This rule is useful for security administrators monitoring web applications and database-backed websites. Analysts should investigate the source address, requested URL, affected application, and server logs. Developers should also use parameterised queries and proper input validation to reduce SQL injection risks.

Rule Docs 1:13513

Rule 2: SQL injection using comment characters in a URL

Source: Snort Rule 1:19438

Alert message: SQL URL ending in comment characters — possible SQL injection attempt

Summary (approximately 200 words)

This rule is intended to detect possible SQL injection attacks by examining HTTP URLs for suspicious combinations of SQL statements and comment characters. In SQL, the double hyphen (--) can indicate the beginning of a comment, depending on the database engine and syntax. Attackers may attempt to use comment characters to alter the interpretation of a database query and bypass parts of its intended logic.

According to the Snort rule documentation, this signature looks for the double-hyphen sequence alongside SQL statements such as SELECT, UPDATE, or INSERT in the HTTP URI. The URI is the resource identifier requested by the client, which can include a path and query parameters. These locations may contain user-supplied values that an insecure web application passes into a database query.

The detection logic combines suspicious URL content with recognisable SQL-related patterns. When the required conditions match, Snort raises an alert for investigation. This approach helps identify potential attacks without requiring the IDS to know whether the target application is actually vulnerable.

However, the presence of SQL keywords or comment characters does not automatically mean an attack is occurring. Legitimate requests may contain similar text. Therefore, analysts should review the complete request, correlate it with application logs, and confirm whether the target application uses the supplied input in database queries.

Rule Docs 1:19438

Rule 3: Cacti web application SQL injection

Source: Snort Rule 1:65833

Alert message: SERVER-WEBAPP Cacti automation_graph_rules.php SQL injection attempt

Summary (approximately 200 words)

This rule is designed to identify attempts to exploit an SQL injection vulnerability in Cacti, a web-based network monitoring and graphing application. The Snort rule database associates this signature with CVE-2024-31445, a vulnerability involving the application's graph automation functionality. SQL injection can occur when an application includes attacker-controlled input in a database query without handling that input safely.

The signature focuses on HTTP requests directed at the /cacti/automation_graph_rules.php endpoint. In particular, the documented detection examines SQL injection metacharacters in the filter parameter. Metacharacters are special symbols that may change how a query is interpreted when unsafe input is incorporated into SQL statements.

The rule's logic is more targeted than a generic SQL injection signature because it identifies a particular application endpoint and parameter. This helps security monitoring distinguish suspicious requests associated with the known vulnerability from unrelated network traffic. When the required patterns are detected, Snort generates an alert for the security team.

Administrators can use the alert to investigate the request source, review Cacti access logs, check the installed software version, and determine whether exploitation may have occurred. They should also update affected software to a fixed version and restrict unnecessary access to the management interface. As with other signatures, an alert indicates a possible exploit attempt rather than confirmed compromise.



Rule Docs 1:65833

4. Conclusion

Snort rules help security professionals detect malicious network traffic through pattern matching and contextual conditions. The three rules demonstrate different approaches to detecting SQL injection: identifying suspicious GET parameters, recognising SQL statements combined with URL comment characters, and targeting a known vulnerable endpoint in a specific application.

Effective rule analysis requires understanding the rule header, inspected traffic, matching conditions, and alert message. Although signatures provide valuable early warnings, they cannot guarantee that every attack will be detected or that every alert represents a successful attack. Combining Snort with secure application development, software updates, and log analysis provides stronger protection.

References

1. Snort. The Basics — Snort 3 Rule Writing Guide. Official documentation.
2. Snort. Rule Options — Snort 3 Rule Writing Guide. Official documentation.
3. Snort. SQL generic SQL insert injection attempt — Rule 1:13513. Rule documentation.
4. Snort. SQL URL ending in comment characters — Rule 1:19438. Rule documentation.
5. Snort. Cacti SQL injection attempt — Rule 1:65833. Rule documentation.

2. HIDS vs. NIDS Analysis

1. Introduction

Intrusion Detection Systems (IDS) are security tools that monitor computer systems or network traffic to identify suspicious activities, policy violations, and possible cyberattacks. Two common types are Host-Based Intrusion Detection Systems (HIDS) and Network-Based Intrusion Detection Sys

tems (NIDS).

- HIDS (Host-Based Intrusion Detection System): Monitors activities on an individual computer or server, including system logs, file changes, running processes, and user activity.
- NIDS (Network-Based Intrusion Detection System): Monitors network traffic passing through a network segment or monitoring point to identify suspicious communication, attack patterns, and unauthorised access attempts.

Both improve cybersecurity, but they differ in what they monitor, where they are installed, and the threats they can detect.

2. Detailed comparison table

Comparison factor	HIDS	NIDS
Full name	Host-Based Intrusion Detection System	Network-Based Intrusion Detection System
Monitoring location	Installed on individual servers or computers	Positioned at strategic network points
Main purpose	Detects suspicious activity on a specific host	Detects suspicious activity in network traffic
Monitoring scope	One host at a time	Multiple devices communicating through a monitored network
Data analysed	System logs, file integrity, processes, user activity and configuration changes	Network packets, protocols, connection patterns and traffic flows
Installation	Requires an agent or software on each monitored host	Requires access to network traffic through a suitable network interface, TAP or SPAN port
Attack visibility	Can detect suspicious local activity and changes after access is gained	Can detect network attacks and suspicious communication visible at its monitoring point
Encrypted traffic	Can inspect host-level information after decryption, depending on its capabilities	Payload inspection is limited when traffic is encrypted and not decrypted before monitoring
Resource usage	Uses host CPU, memory and storage	Uses resources on the monitoring appliance or sensor
Examples	Wazuh agent, OSSEC, Tripwire	Snort, Suricata, Zeek
Best use	Monitoring server integrity, logs and suspicious local actions	Monitoring network threats, scans and malicious traffic

3. Advantages and disadvantages of HIDS and NIDS

HIDS: Host-Based Intrusion Detection System

Advantages	Disadvantages

1. Monitors system logs for suspicious events.	1. Must be installed and maintained on each host.
2. Detects unauthorised changes to important files.	2. Consumes server CPU, memory and storage.
3. Can identify suspicious user activity and local processes.	3. May affect performance on resource-limited servers.
4. Can provide visibility into activity occurring after traffic is decrypted on the host.	4. A compromised host may allow attackers to tamper with the agent or its logs.
5. Helps investigate incidents using detailed host-level evidence.	5. Cannot independently monitor all network devices and traffic.
6. Supports file-integrity monitoring and configuration checks.	6. Requires regular updates, configuration and alert management.
7. Can detect some attacks that leave little visible network evidence.	7. Managing agents across many servers can become complex.

NIDS: Network-Based Intrusion Detection System

Advantages	Disadvantages
1. Monitors traffic from multiple devices at a suitable network location.	1. Cannot automatically see traffic that does not pass its monitoring point.
2. Detects suspicious scans, connections and known attack patterns.	2. Encrypted traffic limits inspection of packet contents.
3. Can identify repeated attack attempts against exposed services.	3. High traffic volumes may overwhelm the sensor.
4. Does not require an agent on every monitored device.	4. Usually provides less visibility into local processes and file changes.
5. Can detect communication with known malicious destinations.	5. May generate false positives from legitimate traffic.
6. Offers centralised network monitoring.	6. Correct placement and network configuration are essential.
7. Can support incident investigation through packet and connection records.	7. May miss attacks that use unfamiliar techniques or evade its rules.

Note: Both systems primarily detect suspicious activity. Blocking attacks requires a prevention capability or additional security controls.

4. Recommendation for a small e-commerce business (300 words)

Recommendation: Prioritise HIDS on the web server first.

A small e-commerce business should prioritise deploying a Host-Based Intrusion Detection System (HIDS) on its web server first, especially if it can only afford one solution initially. An e-commerce server processes customer information, manages product listings, handles orders, and may interact with payment services. Protecting the server against unauthorised access and changes is therefore essential.

HIDS provides direct visibility into activities occurring on the web server. It can monitor system logs, user accounts, running processes, configuration changes, and important files. For example, if an attacker exploits a vulnerable website and modifies its checkout page to steal customer information, a properly configured HIDS may detect suspicious file changes or unusual system activity. It can also help administrators investigate unauthorised login attempts and identify signs of malware or privilege escalation.

Another advantage is that HIDS can examine host-level information that a Network-Based Intrusion Detection System (NIDS) might not see. This is particularly useful when malicious activity occurs after an attacker has gained access to the server or when encrypted network traffic prevents detailed inspection by a NIDS sensor.

However, HIDS has limitations. It consumes server resources, requires maintenance, and may be compromised if an attacker gains administrative control. It also cannot provide comprehensive visibility into network attacks affecting other devices.

For these reasons, the business should deploy HIDS first, configure centralised logging, monitor critical files, and establish alerts for suspicious activity. It should also apply security updates, enforce multi-factor authentication for administrative accounts, use a web application firewall, and maintain secure backups.

Once its budget allows, the business should add a NIDS to monitor incoming and outgoing traffic, detect port scans, and identify suspicious network connections. Combining both systems provides defence in depth: HIDS focuses on the server itself, while NIDS examines network activity. This staged approach offers practical protection for a small business while allowing its security infrastructure to grow as the company expands.

5. Conclusion

HIDS and NIDS perform complementary roles in cybersecurity. HIDS provides detailed information about activity on individual systems, while NIDS provides broader visibility into network communications. For a small e-commerce business with limited resources, deploying HIDS on the web server first is a reasonable starting point, followed by NIDS as resources permit. Using both alongside firewalls, secure authentication, updates, and backups provides stronger overall protection.

3. Case Study: The 2013 Target Data Breach

Analysis — approximately 400 words

Introduction

The 2013 Target data breach was one of the major cybersecurity incidents affecting a large American retailer. During November and December 2013, cybercriminals gained access to Target's network and stole payment card information and personal information belonging to as many as 110 million customers. A US Senate staff report identified several missed opportunities to detect and stop the attack, including failures to respond to alerts generated by security software. [1]

Role of the Intrusion Detection System

Target had deployed a FireEye malware detection system designed to identify suspicious software and malicious activity. The system reportedly generated urgent alerts when attackers installed malware associated with the theft and transfer of customer data. Target's Symantec antivirus software also detected suspicious activity on a server involved in the incident. However, these warnings did not lead to an effective response, and the automatic malware-removal capability of FireEye was reportedly disabled. [1][2]

The attackers had initially obtained access through credentials associated with a third-party HVAC vendor. They subsequently moved through Target's network, compromised systems involved in payment processing, and used malware to collect payment card data. The stolen information was transferred outside the company's network. The intrusion detection system therefore played an important role: it provided warnings that could have helped security personnel identify the attack before more information was stolen.

Importance of Responding to Alerts

The incident demonstrates that detecting an attack is not the same as preventing it. An IDS can identify suspicious behaviour, but its effectiveness depends on whether people or automated controls investigate and respond to its warnings. If alerts are ignored, even an advanced security system may fail to prevent significant damage.

Organisations should establish a formal incident-response process. Alerts must be prioritised according to severity, assigned to responsible personnel, investigated promptly, and escalated when necessary. High-confidence malware detections should trigger containment measures, such as isolating affected systems, blocking malicious connections, and preserving evidence for forensic investigation. Automated blocking or

removal should be enabled where appropriate and tested carefully to avoid disrupting legitimate operations.

Businesses must also reduce alert fatigue through effective tuning, staff training, clear responsibilities, and regular response exercises. Monitoring should operate continuously, with documented procedures and measurable response times.

Conclusion

The Target breach shows that cybersecurity depends on both technology and human decision-making. FireEye reportedly detected important warning signs, but the failure to act allowed the attack to continue. A reliable alert-response process, supported by trained staff, appropriate automation, network segmentation, and regular security reviews, is essential for turning intrusion detection into effective protection.

References

[1] US Senate Committee on Commerce, Science, and Transportation. *A "Kill Chain" Analysis of the 2013 Target Data Breach* (2014). https://www.commerce.senate.gov/services/files/24d3c229-4f2f-405d-b8db-a3a67f183883

[2] SC Media. *Target did not respond to FireEye security alerts prior to breach, according to report* (March 13, 2014). https://www.scworld.com/news/target-did-not-respond-to-fireeye-security-alerts-prior-to-breach-according-to-report